

[ALL] notes

1. Who are the key stakeholders in this process, and what are their most critical needs for consumer intelligence?
 - a. Market Operations
 - i. Products
 - ii. Brand Communication
 - b. RnD
 - i. Packaging
 - ii. Promo
 - c. Product Supply
2. What kind of data products can be created?
 - a. Dashboard
 - b. Sentiment Analysis (Positive/Negative)
 - c. Decision Support
 - i. ARM (Association Rule Mining), better categories
3. How does the E2E data flow look like from web scraping to analytics?
 - a. End to end
4. How do you find quality insights in the large and dirty dataset?
 - a.

Main Problem Statement: Analytics is not fully utilized for actionable insights because consumer and market data is fragmented, high in volume, and stored in multiple formats, making it difficult to generate timely, reliable, and department-specific intelligence. This affects key departments such as Market Operations, R&D, and Product Supply, which each require different kinds of insights to support prioritization and decision-making. The problem is further made worse by multilingual data, jargon, sarcasm, AI hallucinations, and feedback biases such as the gap between stated and revealed preferences, social desirability bias, and the loud minority effect.

1. Analytics is not fully utilized in actionable insights due to fragmented analytics and due to high volume of data entries in multiple formats.
 - a. How to reinforce human discretionary action for specific departments?
 - b. Market Operations
 - i. Needs fast visibility on what is happening in the market right now
 - ii. Location based insights
 - iii. Prioritization? Needs to know what issues need immediate action at store, distributor, or field level
 - iv. Clearer Recommendations
 - c. R&D
 - i. Needs deeper insight into product complaints and unmet needs
 - ii. Wants structured feedback on formulation, scent, texture, durability, performance, and usability
 - iii. Needs evidence for product improvement decisions
 - iv. Cares about separating real product issues from noise

- d. Product Supply
 - i. Needs to know what customers like, dislike, and repeatedly mention
 - ii. Wants feature-level and formulation-level feedback
 - iii. Needs signals on product performance across variants and brands
 - iv. Cares about which product issues are most urgent to fix
- 2. Difficulty in interpreting multiple languages and jargon.
 - a. Sarcasm
 - b. AI Hallucinations
 - c. Data often captures "stated preference" rather than "revealed preference."
 - **Social Desirability Bias:** In surveys, customers often report that they prefer sustainable or "healthy" options because it aligns with their self-image, but their transaction data shows they still prioritize price and convenience.
 - **The "Loud Minority" Effect:** Social media sentiment analysis is often skewed by a tiny percentage of highly vocal users. This can lead brands to over-correct based on the opinions of people who might not even be their core customers.

Solutions

- Dashboard: Unified consumer intelligence layer (a platform, we'll need a sample concept)
 - Reviews
 - Surveys
 - Social Media
 - Customer service feedback
 - Sales / transaction data
 - Product meta data
 - Brand and competitor references
- Sentiment Analysis (Positive/Negative)
 - What's wrong with the product/shop? (Formulation, Description—for each brand)
 - For instance: Formulation: matapang > tide
 - Packaging: too little, too expensive, value for money
- Decision Support
 - Process optimization
 - Presenting it to the different stakeholders with different insights
 - Product demand forecasting

Raw data view, role based dashboards, configurable dashboards, data scraping

Scrape, to clean, publish data link

Agentic

- Self healing agent

- ETL
- Looking to improve themselves
- Ex. new ads new attributes. Without interfering ETL. An agent can fix this
- Agentic Analytics
- Chatbot

Tech stack

- Python
- CSV (MySQL)
- Figma
- React

jn notes

Subcase

1. Digital Product Manager
 - Project Charter
2. Data Engineer
 - Extract Transform Load
 - Tools and Architecture used to do the data processing
 - Presented through a Dashboard or consolidated data set
 -
3. AI Engineer
4. Site Reliability Engineer
 - Resilience engineering > products are reliable and resilient

20 minutes presentationllllllll

5 minutes Q&A

Presentation

- CAR
- What are we trying to solve, how are we doing it, why are we doing it
- All information should be in the PPT

[Migui][Complete] Operations Manager

SRE Strategy Document and Incident Response Playbook

Purpose. This document defines the reliability strategy, service levels, monitoring model, and incident response process for a consumer insight analytics platform that ingests Lazada and related feedback data into Azure-based storage, processing, AI, API, and dashboard layers.

Reliability objective. Keep ingestion, ETL, AI processing, APIs, and dashboards available, fresh, and trustworthy so Market Operations, R&D, and Product Supply can act on timely signals instead of stale, delayed, or corrupted outputs (Google, n.d.; Microsoft Learn, 2025–2026).

Priority	SLI	Definition	Monthly SLO	Why it matters
P1	Pipeline Success Rate	successful scheduled runs ÷ total scheduled runs	>= 99.0%	Prevents missed refreshes and broken publish cycles
P2	Data Freshness	critical datasets published within 2 hours ÷ total critical datasets	>= 95.0%	Prevents stale dashboards and delayed decisions
P3	Analytics Availability	successful dashboard/API requests ÷ total dashboard/API requests	>= 99.5%	Keeps the product usable for stakeholders

Monitoring and alerting strategy. Azure Monitor is the central observability layer; Application Insights tracks web and API latency, dependency calls, and failures; Log Analytics stores and queries logs; Azure Monitor Alerts and Action Groups route notifications and automated actions; Azure Data Factory and Azure Storage metrics are monitored for ETL and data lake reliability (Microsoft Learn, 2025–2026).

- **Source and ingestion:** scraper or API success rate, source latency, missing file count, schema drift, and source completeness
- **ETL and processing:** job failure rate, retry count, null rate, duplicate rate, validation pass rate, and publish delay
- **AI and analytics:** malformed JSON rate, fallback rate, token usage anomaly, uncategorized volume, confidence-score drop, and prompt degradation signals
- **Delivery:** dashboard availability, widget failure rate, page response time, and stale-data indicator breaches
- **Alert routing:** P1 and P2 use Azure Action Groups with email, SMS, and webhook-based escalation; P3 to P5 use email plus ticket creation. Safe self-healing actions may retry jobs, restart stateless workers, quarantine bad batches, or switch to the last known good dataset. Azure Action Groups support notifications and automated actions such as email, SMS, webhooks, Azure Functions, Logic Apps, and ITSM integration (Microsoft Learn, 2025).

Severity and support model. P1 = critical outage, 24x7, acknowledge in 15 minutes, restore target 3 hours; P2 = major degradation, 24x7, acknowledge in 30 minutes, restore target 8 hours; P3 = moderate issue with workaround, 15x5, acknowledge in 4 business hours, restore target 2 business days; P4 = minor issue, 15x5, acknowledge in 1 business day, restore in next fix window; P5 = informational or backlog item, business hours, handled through the improvement backlog. (Google, n.d.; NIST, 2025).

SRE Strategy Document and Incident Response Playbook

Incident points across the pipeline. Incidents can occur at source or API ingestion, data lake landing, ETL and validation, AI inference, API service, or dashboard delivery. The table below defines incident criteria, detection rules, automatic actions, and manual handoff conditions.

Stage	Typical Incident	Detection rule	Auto action	Manual handoff rule
Source or API	Timeout, auth failure, source layout change	2 failed pulls or source completeness below 95%	retry up to 3 times	escalate to Data Engineer if critical feed still fails
Data lake landing	Missing file, corrupt file, wrong partition	freshness lag above SLA or checksum failure	hold publish; keep last good batch	manual review if no safe fallback is available
ETL or validation	Null spike, duplicate spike, schema drift	null rate above 5%, duplicate rate above 1%, or required-column mismatch	quarantine failed batch; rerun job	manual fix if schema change affects required fields
AI inference	Malformed JSON, high fallback rate, token spike	fallback rate above 10% or malformed JSON above 3%	retry once with safe prompt; suppress low-confidence output	escalate to AI Engineer if issue repeats or affects critical alerts
Dashboard or API	5xx spike, slow response, stale dashboard	error rate above 2%, p95 above 2 seconds, or freshness breach	restart app or roll back last deployment	manual intervention if outage persists over 15 minutes

Response workflow. 1) Detect through alert, health check, or user report. 2) Triage severity, scope, and affected stakeholders. 3) Contain by pausing publish, isolating bad data, or rolling back a release. 4) Recover by rerunning jobs, restoring service, or switching to fallback data. 5) Validate freshness, availability, and data quality before closing. 6) Log the incident and open an RCA-CAP item for prevention. This workflow follows the preparation, detection, analysis, containment, recovery, and improvement principles reflected in NIST incident-response guidance (NIST, 2025).

Notification model. Azure Action Groups send P1 and P2 alerts by email, SMS, and webhook-based escalation. The Operations Manager owns the incident bridge; the Data Engineer leads data pipeline repair; the AI Engineer leads model-output repair; the Project Manager sends business-facing status updates when the incident affects stakeholder timelines. P3 to P5 are routed through email and ticket creation based on business impact (Microsoft Learn, 2025).

Incident severity model. This playbook uses the same P1 to P5 severity definitions and response targets defined in the SRE Strategy Document.

SRE Strategy Document and Incident Response Playbook

Task	Ops Mgr	Data Eng	AI Eng	Proj Mgr
Detect alert and declare severity	A/R	C	C	I
Open incident bridge and assign owner	A/R	C	C	I
Pause publish and enable fallback dataset	A	R	C	I
Restore ingestion or source connection	A	R	I	I
Restore ETL or validation job	A	R	I	I
Restore AI extraction, prompt, or output logic	A	I	R	I
Validate dashboard freshness and usability	A	R	C	I
Send incident status updates	R	I	I	A
Approve incident closure	A/R	C	C	I
Complete RCA and corrective action plan	A	R	R	C

Legend: R = Responsible, A = Accountable, C = Consulted, I = Informed.

Root-cause analysis and corrective action plan. Every P1 and P2 incident requires an RCA within 2 business days and a corrective action plan within 5 business days. The RCA must capture the incident timeline, trigger, failed control, why the issue was not detected earlier, user impact, and the control gap. The CAP must assign an owner, due date, and validation metric such as lower repeat-alert rate, lower null-rate variance, or improved freshness compliance. This reflects the NIST emphasis on improving detection, response, and recovery effectiveness after incidents (NIST, 2025).

Risk assessment levels. Level 1 = high risk; direct impact on dashboard availability, critical feed freshness, or data integrity. Level 2 = medium risk; partial degradation with workaround. Level 3 = low risk; localized defect or informational alert. For this dataset, key resilience concerns are schema drift from marketplace pages, multilingual and Taglish misclassification, review spam, missing competitor mappings, and accidental exposure of personal data. Controls include schema contracts, confidence thresholds, quarantine tables, human review for low-confidence AI outputs, and secure secret handling in Azure Key Vault. NIST's AI RMF supports the use of trust, governance, and risk controls for AI systems, while Azure Key Vault is designed for secure storage and access to secrets, keys, and certificates (NIST, 2023; Microsoft Learn, 2025–2026).

Backlog and release management. Improvements are prioritized by **(business impact × recurrence × detectability gap) ÷ implementation effort**. Priority 1 backlog items: freshness alerts, publish gates, fallback dataset, and dashboard uptime checks. Priority 2: schema-drift rules, AI confidence gates, and duplicate or null controls. Priority 3: automated retries, runbook automation, synthetic health-check scripts, and capacity dashboards. Releases should use a controlled change window, rollback plan, health-check script, and post-release validation of freshness, availability, and AI output quality before full promotion.

[1] SRE Strategy Document

SRE Strategy Document

Purpose. This document defines the reliability strategy, service levels, monitoring model, and incident response process for a consumer insight analytics platform that ingests Lazada and related feedback data into Azure-based storage, processing, AI, API, and dashboard layers.

Reliability objective. Keep ingestion, ETL, AI processing, APIs, and dashboards available, fresh, and trustworthy so Market Operations, R&D, and Product Supply can act on timely signals instead of stale, delayed, or corrupted outputs (Google, n.d.; Microsoft Learn, 2025–2026).

Priority	SLI	Definition	Monthly SLO	Why it matters
P1	Pipeline Success Rate	successful scheduled runs ÷ total scheduled runs	>= 99.0%	Prevents missed refreshes and broken publish cycles
P2	Data Freshness	critical datasets published within 2 hours ÷ total critical datasets	>= 95.0%	Prevents stale dashboards and delayed decisions
P3	Analytics Availability	successful dashboard/API requests ÷ total dashboard/API requests	>= 99.5%	Keeps the product usable for stakeholders

Monitoring and alerting strategy. Azure Monitor is the central observability layer; Application Insights tracks web and API latency, dependency calls, and failures; Log Analytics stores and queries logs; Azure Monitor Alerts and Action Groups route notifications and automated actions; Azure Data Factory and Azure Storage metrics are monitored for ETL and data lake reliability (Microsoft Learn, 2025–2026).

- **Source and ingestion:** scraper or API success rate, source latency, missing file count, schema drift, and source completeness
- **ETL and processing:** job failure rate, retry count, null rate, duplicate rate, validation pass rate, and publish delay
- **AI and analytics:** malformed JSON rate, fallback rate, token usage anomaly, uncategorized volume, confidence-score drop, and prompt degradation signals
- **Delivery:** dashboard availability, widget failure rate, page response time, and stale-data indicator breaches
- **Alert routing:** P1 and P2 use Azure Action Groups with email, SMS, and webhook-based escalation; P3 to P5 use email plus ticket creation. Safe self-healing actions may retry jobs, restart stateless workers, quarantine bad batches, or switch to the last known good dataset. Azure Action Groups support notifications and automated actions such as email, SMS, webhooks, Azure Functions, Logic Apps, and ITSM integration (Microsoft Learn, 2025).

Severity and support model. P1 = critical outage, 24x7, acknowledge in 15 minutes, restore target 3 hours; P2 = major degradation, 24x7, acknowledge in 30 minutes, restore target 8 hours; P3 = moderate issue with workaround, 15x5, acknowledge in 4 business hours, restore target 2 business days; P4 = minor issue, 15x5, acknowledge in 1 business day, restore in next fix window; P5 = informational or backlog item, business hours, handled through the improvement backlog. (Google, n.d.; NIST, 2025).

[2] Incident Response Playbook

Incident Response Playbook

Incident points across the pipeline. Incidents can occur at source or API ingestion, data lake landing, ETL and validation, AI inference, API service, or dashboard delivery. The table below defines incident criteria, detection rules, automatic actions, and manual handoff conditions.

Stage	Typical Incident	Detection rule	Auto action	Manual handoff rule
Source or API	Timeout, auth failure, source layout change	2 failed pulls or source completeness below 95%	retry up to 3 times	escalate to Data Engineer if critical feed still fails
Data lake landing	Missing file, corrupt file, wrong partition	freshness lag above SLA or checksum failure	hold publish; keep last good batch	manual review if no safe fallback is available
ETL or validation	Null spike, duplicate spike, schema drift	null rate above 5%, duplicate rate above 1%, or required-column mismatch	quarantine failed batch; rerun job	manual fix if schema change affects required fields
AI inference	Malformed JSON, high fallback rate, token spike	fallback rate above 10% or malformed JSON above 3%	retry once with safe prompt; suppress low-confidence output	escalate to AI Engineer if issue repeats or affects critical alerts
Dashboard or API	5xx spike, slow response, stale dashboard	error rate above 2%, p95 above 2 seconds, or freshness breach	restart app. or roll back last deployment	manual intervention if outage persists over 15 minutes

Response workflow. 1) Detect through alert, health check, or user report. 2) Triage severity, scope, and affected stakeholders. 3) Contain by pausing publish, isolating bad data, or rolling back a release. 4) Recover by rerunning jobs, restoring service, or switching to fallback data. 5) Validate freshness, availability, and data quality before closing. 6) Log the incident and open an RCA-CAP item for prevention. This workflow follows the preparation, detection, analysis, containment, recovery, and improvement principles reflected in NIST incident-response guidance (NIST, 2025).

Notification model. Azure Action Groups send P1 and P2 alerts by email, SMS, and webhook-based escalation. The Operations Manager owns the incident bridge; the Data Engineer leads data pipeline repair; the AI Engineer leads model-output repair; the Project Manager sends business-facing status updates when the incident affects stakeholder timelines. P3 to P5 are routed through email and ticket creation based on business impact (Microsoft Learn, 2025).

Incident severity model. This playbook uses the same P1 to P5 severity definitions and response targets defined in the SRE Strategy Document.

Incident Response Playbook

Task	Ops Mgr	Data Eng	AI Eng	Proj Mgr
Detect alert and declare severity	A/R	C	C	I
Open incident bridge and assign owner	A/R	C	C	I
Pause publish and enable fallback dataset	A	R	C	I
Restore ingestion or source connection	A	R	I	I
Restore ETL or validation job	A	R	I	I
Restore AI extraction, prompt, or output logic	A	I	R	I
Validate dashboard freshness and usability	A	R	C	I
Send incident status updates	R	I	I	A
Approve incident closure	A/R	C	C	I
Complete RCA and corrective action plan	A	R	R	C

Legend: R = Responsible, A = Accountable, C = Consulted, I = Informed.

Root-cause analysis and corrective action plan. Every P1 and P2 incident requires an RCA within 2 business days and a corrective action plan within 5 business days. The RCA must capture the incident timeline, trigger, failed control, why the issue was not detected earlier, user impact, and the control gap. The CAP must assign an owner, due date, and validation metric such as lower repeat-alert rate, lower null-rate variance, or improved freshness compliance. This reflects the NIST emphasis on improving detection, response, and recovery effectiveness after incidents (NIST, 2025).

Risk assessment levels. Level 1 = high risk; direct impact on dashboard availability, critical feed freshness, or data integrity. Level 2 = medium risk; partial degradation with workaround. Level 3 = low risk; localized defect or informational alert. For this dataset, key resilience concerns are schema drift from marketplace pages, multilingual and Taglish misclassification, review spam, missing competitor mappings, and accidental exposure of personal data. Controls include schema contracts, confidence thresholds, quarantine tables, human review for low-confidence AI outputs, and secure secret handling in Azure Key Vault. NIST's AI RMF supports the use of trust, governance, and risk controls for AI systems, while Azure Key Vault is designed for secure storage and access to secrets, keys, and certificates (NIST, 2023; Microsoft Learn, 2025–2026).

Backlog and release management. Improvements are prioritized by **(business impact × recurrence × detectability gap) ÷ implementation effort**. Priority 1 backlog items: freshness alerts, publish gates, fallback dataset, and dashboard uptime checks. Priority 2: schema-drift rules, AI confidence gates, and duplicate or null controls. Priority 3: automated retries, runbook automation, synthetic health-check scripts, and capacity dashboards. Releases should use a controlled change window, rollback plan, health-check script, and post-release validation of freshness, availability, and AI output quality before full promotion.

migui notes

A data Reliability Engineer DRE is a specialized role that merges principles from data engineering and site reliability engineering SRE, ensuring the

Reliability
Availability,
And performance

Of the **data systems** and **pipelines** throughout the **data lifecycle**

As the DRE, provide an incident response plan to determine the **root cause of potential incidents** and its resolution

Moreover, as a part of transitioning from firefighting to resilience engineering.

Present a strategy to enhance the reliability and performance of your current customer insights product based on the identified potential incidents.

the pdf is context and the image shows written comments please read it if ever here are what I wrote: SRE Strategy Document SLIs which are the P1 P2 P3 SLI & SLO Monitoring (What are we monitoring?) and alerting strategy (adding self healing, dashboard strategy) Incident response Playbook How to automate and prevent issues (proactive operations) this could be the Dashboard, with alerting notificationss specify how people are being alerted if its thru SMTP or text messages, this is in the context of using Azure but we're building a webpage front end with azure data lakes and such you get the gist. for the data pipeline we need the crietrias for incidents resolving the issue we will do action plan (this can involve both the self-healing feature and if its beyond its control there should be a criteria wherein it should handoff to manual intervention) key stakeholders involved it should very specific using RACI there should be categories with multiple tasks and the the role/stakholder columns themselves typically we should also have a RCA Prcoess to establish that issues does not occur again and what are our precautionary measures we need to be very specific CAP or context action protocol.. answering that what SLIs and SLOs will ensure that a similar incident will not occur again How do we plan to monitor these metrics Based on the dataset what are some other resilience concerns we can forsee in the rest of the product, how do we intend to address these concerns so this is the Risk Assesment portion, we need to be able to visualize and be specific about this Lastly what is our prioritization plan for all of these improvments, so a Backlog Management of some sort for improvement (if you have ideas on what could be done here, please do them) so 1 page for SRE Strategy Document and 2 pages for Incident Response Playbook.. make sure ALWAYS there is a reference and reasoning for every word and every technical speicifc word or calculation involved.file:///D:/Downloads/Team8-PG-Next.docx i need the expected outputs, please give me the WORD document format and then take your time and giving it to me for copy and paste as well

Question 1 for operations we're looking to do self-healing yes. Is Gemini API a good method?
Question 2 are we allowed to include an extra page for references and glossary

Day 1

CAR

Context Action Result

Role 1 Outputs

Product Vision & Strategy Document 1 page

Product Features & Roadmap 1 page

Role 2: Data Engineer

ETL extract transform load

Data Pipeline Architecture 1 page

Data Output (tangible, clean visuals)

Analytics Methodology (data cleaning, validation, and transformation)

- Has references

Recommendations

Python

Google colab

Github copilot

Excel

PowerBI

Draw.io

AI engineer

- Generative, chatbots, etc.. summarize the rational

AI Model Architecture 1 Page

AI methodology - Summary Report

Model Training

GEN AI Tools

Cloud Tools

Potential AI Applications

Sentiment Analysis Positive/Negative

Review Feedback Classification Including Spam Unrelated

Brand Competitor Insights SUMmarizairtion

Product Review Image Analysis

COmpany Brand-Product mapping clarification

Operations Manager

SRE Strategy Document 1 page

SLIs and SLOs 3 each (P1, P2, P3)

24/7 15/5

Data analysis 15x5 L1

Data bridge data integrity data missing. (what were monitoring data)

Monitoring and altering Strategy

Incident Response Playbook 2 page

How to automate and prevent issues 2 pages

RACI

Criteria for incident; type of error capability to notify

Self healing (automation)

Root cause analysis process

Correct action plan

Risk Assessment level 1 level 2 level 3

Backlog management

Release Management

if not self healing let us do Automatic Health Checks Scripting

End of lifecycle

Incident - p1 3 hours

Request p1 2 days

arbitrary number

OKAY

- Let's work on making BPMNs to show the process of the incident like what each role does step by step.

Sample

SRE Strategy Document and Incident Response Playbook

Consumer Intelligence Platform

Purpose. This document defines the reliability strategy, service levels, monitoring approach, severity handling, RACI ownership, and incident response procedures for a customer insights / consumer intelligence platform that ingests web, social, survey, customer service, and sales data.

1. Platform Context

The platform collects high-volume, multilingual, and multi-format consumer data from reviews, surveys, social media, customer service feedback, sales or transaction records, product metadata, and brand or competitor references. It then transforms those inputs into dashboards, sentiment outputs, categorization results, forecasting signals, and decision-support views for functions such as Market Operations, R&D, Product Supply, Brand Communication, Packaging, and Promo.

Because stakeholders rely on fresh and trustworthy outputs, the operating model must protect the full chain: source collection, ETL, model inference, storage, APIs, dashboards, and chatbot or assistant experiences. The SRE goal is to move from reactive firefighting to resilience engineering through stronger monitoring, alerting, validation, automation, and recovery controls.

2. Reliability Objectives

The reliability strategy focuses on four practical objectives. First, keep ingestion and ETL stable so source failures, schema drift, and malformed records do not silently break downstream analytics. Second, protect the quality and trustworthiness of analytics outputs by stopping stale, duplicated, low-confidence, or clearly corrupted data before publish. Third, keep dashboards, APIs, and stakeholder-facing tools available and responsive. Fourth, reduce repeated incidents through proactive controls such as automated retries, fallback datasets, quarantine tables, self-healing workflows, and clear ownership during incidents.

3. Service Level Indicators and Objectives

The platform will track three core SLIs and their matching SLOs.

Metric	Definition	Target SLI	Monthly SLO
Pipeline Success Rate	Percent of scheduled ingestion and ETL jobs that complete successfully.	Job outcome monitoring	99.0% successful runs
Data Freshness	Elapsed time between source arrival and availability in processed outputs.	Freshness by stage	95.0% within 2 hours
Analytics Availability	Percent of time dashboards, APIs, and decision-support services are accessible.	Availability and error monitoring	99.5% uptime

4. Monitoring and Alerting Strategy

- Source and ingestion monitors: scraper/API success rate, source latency, missing file detection, schema drift alerts, duplicate spikes, and feed completeness checks.
- ETL and processing monitors: job duration, job failure rate, record counts in vs out, null spikes, transformation errors, mapping failures, and validation pass/fail rates.
- AI and analytics monitors: sentiment drift, classification anomalies, high uncategorized volume, confidence score drops, language-detection failures, and hallucination-risk flags for generated summaries.
- Dashboard and delivery monitors: uptime, response time, stale-data indicators, widget/API errors, broken role filters, chatbot failure rate, and end-user access issues.
- Alerting principles: alerts must be routed by business impact, mapped to a runbook, and assigned to an owner. Noisy alerts should be tuned or merged so they remain actionable.

5. Incident Severity Matrix

Support coverage and response expectations are defined separately for each priority level.

Priority	Definition	Business Impact	Initial Response	Status Update	Coverage
P1	Critical outage	Full platform or critical pipeline unavailable; no acceptable workaround.	15 min	Every 30 min	24x7
P2	Major incident	Large business impact or one major service severely degraded.	30 min	Every 1 hr	24x7
P3	Moderate incident	Partial degradation with workaround available; limited but real impact.	4 business hrs	Every 4 business hrs	15x5
P4	Minor incident	Low urgency issue with small impact or minor defect.	1 business day	Daily / as needed	15x5
P5	Informational / backlog item	No active outage; tracked for improvement or non-urgent fix.	Next planning cycle	As needed	Business hrs

P1 - Critical outage

- Examples: ingestion fully down, dashboard unavailable to all users, critical data corruption, or analytics publish blocked for a business-critical cycle.
- Coverage: 24x7. The Operations Manager opens the incident bridge immediately, assigns technical owners, and starts stakeholder communication.
- Required actions: contain the issue, restore service with fallback or rollback if needed, and protect downstream users from stale or corrupted outputs.
- Communication: update every 30 minutes until recovery, then issue a preliminary summary and schedule a post-incident review.

P2 - Major incident

- Examples: one major source or major dashboard view is severely degraded, freshness breach affects a key department, or model output is materially unreliable.
- Coverage: 24x7. The Operations Manager coordinates triage, while the affected technical owner leads fix execution.
- Required actions: assess business impact quickly, implement a workaround or partial restore, and decide whether escalation to P1 is needed.
- Communication: update every 1 hour until stable, then document root cause, recovery steps, and preventive actions.

P3 - Moderate incident

- Examples: one workflow is degraded, one stakeholder group is affected, or there is a moderate quality problem with a documented workaround.
- Coverage: 15x5. Work is handled within support hours unless impact increases.
- Required actions: validate scope, assign technical owner, restore normal operation, and log any follow-on resilience work.
- Communication: update every 4 business hours and escalate if business impact expands.

P4 - Minor incident

- Examples: small dashboard defect, non-critical alert noise, minor data mismatch with low business impact, or localized access issue.
- Coverage: 15x5.
- Required actions: assign to the responsible team, confirm workaround if needed, and target correction in the normal support window.
- Communication: daily or as needed.

P5 - Informational or backlog item

- Examples: enhancement request, resilience concern, operational debt, tuning work, or non-urgent bug found during review.
- Coverage: business hours.
- Required actions: document the issue, assess risk, and route it into planning or backlog prioritization.
- Communication: only when requested or when status changes materially.

6. RACI Matrix

The matrix below uses Category and Task as the first two columns, followed by the roles involved in delivery and incident handling.

Category	Task	Operations Manager	Data Engineering	AI Engineering	InfoSec	Project Manager	Business Stakeholder	Generic User
Governance	Define SLIs, SLOs, severity criteria, and support model	A/R	C	C	C	C	I	I
Monitoring	Build and maintain pipeline, model, and dashboard monitors	A	R	R	C	I	I	I
Alerting	Tune alert thresholds and routing rules	A/R	R	R	C	I	I	I
Data Quality	Maintain schema checks, null checks, duplicate checks, and publish gates	C	A/R	C	C	I	I	I
AI Quality	Track confidence, drift, hallucination risk, and fallback rules	C	I	A/R	C	I	C	I
Security	Review access controls, secrets handling, and incident	C	C	C	A/R	I	I	I

	security impact							
Release Mgmt	Approve deployment schedule, rollback readiness, and change windows	C	R	R	C	A/R	I	I
Incident Mgmt	Detect, declare, and triage incidents	A/R	C	C	C	I	I	I
Restoration	Restore data flow, dashboard service, or model output	A	R	R	C	I	I	I
Communication	Issue business updates, timeline, and recovery notice	R	I	I	I	A	C	I
Validation	Confirm recovered outputs are usable for business decisions	C	C	C	I	I	A/R	C
Postmortem	Lead root-cause review and assign corrective actions	A/R	C	C	C	C	I	I
Backlog	Prioritize resilience fixes and preventive improvements	R	C	C	C	A	C	I

Legend: R = Responsible, A = Accountable, C = Consulted, I = Informed.

7. Incident Response Playbook

Incident points across the pipeline. Incidents may occur during source scraping or API ingestion, staging and storage, ETL and transformation, AI inference and summarization, or dashboard and chatbot delivery. Common failure modes include schema drift, missing feeds, duplicate ingestion, malformed text encodings, low-confidence model output, stale dashboards, and role-filter errors.

Response workflow. First, detect the issue through alerts, health checks, anomaly detection, or user reports. Second, triage the incident by scope, affected stakeholders, current severity, and whether a safe workaround exists. Third, contain the impact by pausing publishes, isolating corrupted batches, failing over to a last-known-good dataset, or suppressing unreliable AI output. Fourth, investigate logs, job history, source changes, record count mismatches, validation errors, and model confidence trends. Fifth, resolve by fixing the connector, patching ETL logic, rerunning jobs, restoring data, rolling back a model or deployment, or refreshing the dashboard service. Sixth, recover by validating freshness, service accessibility, and business usability before formally closing the incident.

Detection earlier and prevention. Earlier detection depends on source heartbeat checks, freshness monitoring at every pipeline stage, record-count anomaly alerts, schema-change alerts, dashboard staleness indicators, and confidence thresholds for generated outputs. Preventive controls should include publish gates, quarantine tables for bad records, fallback datasets, automated retries, and rollback-ready deployments.

Proactive operations. To reduce repeated incidents, the platform should add self-healing behaviors where safe: retry failed connectors, restart failed workers, route broken records to quarantine rather than blocking the whole pipeline, and suggest mapping updates when new ad terms or product attributes appear. Any agentic or self-healing action must remain bounded by approval rules so the platform does not silently change business logic without review.

Prioritization plan. Priority 1 is stability of the core pipeline, dashboard availability, and freshness monitoring. Priority 2 is data trust through validation gates, mapping checks, and AI confidence controls. Priority 3 is reduction of manual firefighting through retries, quarantine workflows, and standard runbooks. Priority 4 is long-term resilience through drift monitoring, recurring-incident reviews, capacity planning, and backlog governance.

Appendix A. Standards and References Basis

Purpose. This appendix links the SRE strategy and incident response playbook to recognized standards, technical guidance, and relevant research, while also tying the design back to P&G's stated strategy and the fabric-care use case. In this case, the product scope is consumer insight analytics for P&G fabric-care brands using Lazada review data plus other feedback sources.

Why this matters for the case. P&G explicitly frames growth around five vectors of superiority—product, package, brand communication, retail execution, and value—and fabric care is one of its core categories. That makes a role-based consumer intelligence platform operationally relevant because the same review stream can be translated into different actions for Market Operations, R&D, Product Supply, Packaging, and Brand Communication.

A1. Mapping the proposed strategy to standards and articles

Strategy Element	Why it fits the P&G / Lazada fabric-care case	Standards and articles that support it
SLIs and SLOs for pipeline success, freshness, and analytics availability	The platform is only useful if consumer feedback is processed on time and dashboards stay available for business users. Freshness matters because	Google SRE book and workbook support SLIs/SLOs and SLO-based alerting for user-relevant reliability metrics.

Strategy Element	Why it fits the P&G / Lazada fabric-care case	Standards and articles that support it
	marketplace feedback changes quickly and late insights reduce business value.	
Monitoring and alerting centered on business impact	Alerts should fire on what breaks stakeholder decisions: stale dashboards, failed ingestion, bad publishes, and degraded model outputs. This is more practical than alerting only on infrastructure symptoms.	Google SRE workbook recommends alerting from SLOs and keeping SLI metrics prominent on service dashboards.
Incident response lifecycle with triage, containment, recovery, and postmortem	A consumer intelligence platform can fail in scraping, ETL, model inference, storage, APIs, or dashboards. A structured playbook reduces downtime and protects decision quality.	NIST SP 800-61 provides established guidance for incident handling programs, detection, analysis, containment, eradication, and recovery.
Security and privacy controls for consumer feedback data	Reviews, customer-service text, and user-generated content may contain personal data or sensitive details. Logs, datasets, and dashboards should therefore use least privilege and privacy-aware handling.	NIST SP 800-53 Rev. 5 and NIST SP 800-122 support security and privacy controls for information systems and PII handling.
Data quality gates before publish	Because Lazada and other marketplace feeds are noisy, duplicate-prone, and schema-drifting, the system needs null checks, duplicate checks, mapping checks, and quarantine tables before publishing insights.	ISO 8000-1 supports data-quality principles and a structured path to data quality.
Human-in-the-loop controls for AI outputs	The case itself highlights sarcasm, jargon, multilingual text, and hallucination risk. That makes confidence thresholds, fallback rules, and manual review reasonable design choices instead of optional extras.	NIST AI RMF 1.0 and the Generative AI Profile both support managing real-world AI risks, transparency, and governance.
Multilingual and code-mixed sentiment handling	Lazada reviews and social posts in the Philippines commonly mix English with local language, slang, or shortened product descriptors. A multilingual pipeline is therefore necessary, not just nice to have.	Recent ACL work supports multilingual and code-mixed sentiment methods, including lexicon-enhanced and hybrid LLM plus mBERT approaches.
Review spam and synthetic review detection	E-commerce review mining should treat synthetic or deceptive reviews as a quality risk, especially when generative AI can produce convincing spam at scale.	ACL papers on deceptive opinion spam and AI-enhanced opinion spambots support adding spam detection and quality filters.
Cautious use of social data and survey data	Your problem statement already notes the loud-minority effect and the gap between stated and revealed preferences. This supports using social and survey data as one signal among several, not the single source of truth.	Research on online/social-media data as imperfect public-opinion proxies and studies of the intention-search-behaviour gap support this caution.

A2. Notes on how the references apply to your specific design

- **P&G alignment:** Because P&G publicly emphasizes superiority across product, package, brand communication, retail execution, and value, the role-based dashboard concept is not just a technical choice. It maps directly to how the business says it competes, especially in categories such as fabric care.
- **Why freshness is a top SRE metric:** For marketplace and review analytics, stale data can create false comfort. A dashboard that is online but several hours behind can still mislead decision-makers. That is why freshness belongs beside availability and success rate as a core reliability metric.

- **Why quality gates are needed before publishing insights:** Web-scraped and marketplace data often changes structure, contains duplicate text, missing attributes, seller-specific naming, and noisy metadata. Publishing directly from raw ingestion to business dashboards would make the platform fragile and reduce trust.
- **Why AI outputs must stay reviewable:** For sentiment, summarization, and issue classification, a human-review queue is justified when confidence is low or the business impact is high. In this case, that includes possible formulation issues, packaging complaints, safety-related concerns, or priority escalations for Market Operations.
- **Why “stated” and “revealed” preference should be compared:** Survey answers and public posts may say consumers want one thing, while transaction data shows another. For a fabric-care portfolio, that can affect how teams interpret comments about scent, value for money, sustainability, pack size, and convenience.

A3. Reference list

1. Google. (n.d.). Service level objectives. In Site Reliability Engineering. <https://sre.google/sre-book/service-level-objectives/>
2. Google. (n.d.). Alerting on SLOs. In The Site Reliability Workbook. <https://sre.google/workbook/alerting-on-slos/>
3. Google. (n.d.). Monitoring systems with advanced analytics. In The Site Reliability Workbook. <https://sre.google/workbook/monitoring/>
4. National Institute of Standards and Technology. (2021). NIST Special Publication 800-61 Rev. 2: Computer Security Incident Handling Guide. <https://www.nist.gov/privacy-framework/nist-sp-800-61>
5. National Institute of Standards and Technology. (2020). NIST Special Publication 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations. <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
6. National Institute of Standards and Technology. (2010). NIST Special Publication 800-122: Guide to Protecting the Confidentiality of Personally Identifiable Information (PII). https://csrc.nist.gov/files/pubs/sp/800/122/final/docs/sp800_122.epub
7. National Institute of Standards and Technology. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0). <https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf>
8. National Institute of Standards and Technology. (2024). Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile. <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>
9. International Organization for Standardization. (2022). ISO 8000-1:2022 Data quality — Part 1: Overview. <https://www.iso.org/standard/81745.html>
10. Procter & Gamble. (2025). Superiority across product, package, brand communication, retail execution and value. In P&G 2025 Annual Report. <https://us.pg.com/annualreport2025/superiority-across-product-package-brand-communication-retail-execution-and-value/>
11. Procter & Gamble. (2025). Brands. <https://us.pg.com/brands/>
12. Koto, F., et al. (2024). Zero-shot sentiment analysis in low-resource languages using a multilingual sentiment lexicon. Proceedings of EACL 2024. <https://aclanthology.org/2024.eacl-long.18/>
13. Veeramani, H., et al. (2024). Hybrid approaches with large language models for enhanced sentiment analysis in code-switched and code-mixed texts. Proceedings of WILDRE 2024. <https://aclanthology.org/2024.wildre-1.10.pdf>
14. Liyanage, V., et al. (2024). Detecting AI-enhanced opinion spambots: A study on LLM-generated hotel reviews. Proceedings of the 4th Workshop on e-Commerce and NLP. <https://aclanthology.org/2024.ecnlp-1.8/>
15. Mukherjee, A., et al. (2015). Detecting deceptive opinion spam using linguistics, behavioral and relational features. Proceedings of the 2015 International Conference on Weblogs and Social Media. <https://aclanthology.org/P15-5007.pdf>
16. Diaz, F., Gamon, M., Hofman, J. M., Kiciman, E., & Rothschild, D. (2016). Online and social media data as an imperfect continuous panel survey. PLoS ONE, 11(1). <https://pmc.ncbi.nlm.nih.gov/articles/PMC4711590/>
17. Azzopardi, L., et al. (2024). Seeking socially responsible consumers: Exploring the intention-search-behaviour gap. Proceedings of CHIIR 2024. <https://dl.acm.org/doi/abs/10.1145/3627508.3638324>

18. Larson, R. B. (2019). Controlling social desirability bias. *International Journal of Market Research*, 61(5), 534-547. <https://journals.sagepub.com/doi/10.1177/1470785318805305>

References

Keyword explanations. SLI = measured reliability metric; SLO = target value for an SLI; schema drift = unexpected change in columns, required fields, or data types; malformed JSON = AI output that breaks the required structure; last known good dataset = the most recent validated dataset that can be safely served during recovery.

Reference basis for design choices. SLI and SLO definitions follow Google SRE guidance. Incident detection, response, and recovery follow NIST SP 800-61 Rev. 3. Azure monitoring, alerts, and Action Groups follow Microsoft Learn documentation. An SLO is a target value for a service level measured by an SLI.

SRE Companion Appendix: References, Rationale, and Keyword Explanations

Use this appendix for Q&A, panel defense, and technical clarification. The main submission stays concise; this appendix explains why each SRE choice is reasonable for a conceptual Azure-based consumer insight platform.

A. Why these SRE metrics and controls

Why use SLIs and SLOs? Google SRE defines an SLO as a target value measured by an SLI. That is why the document separates the metric itself from the target. Pipeline success rate, freshness, and analytics availability are practical top-level reliability indicators for a data product because they directly affect whether the dashboard is usable and timely.

Why monitor freshness? A dashboard can be online but still misleading if the latest data has not arrived. That is why freshness is treated as a first-class reliability metric instead of just a data engineering detail.

Why use Azure Monitor, Application Insights, and Log Analytics together? Azure Monitor is the unified observability layer, Application Insights provides application performance monitoring for live web apps and APIs, and Log Analytics provides the query layer for logs. This makes the stack coherent for a separate web front end plus Azure data services.

Why use Action Groups? Azure Action Groups define who gets notified and what action is taken when an alert fires. They support email, SMS, push, webhook, and automation actions, so they fit the requirement to specify how people are alerted.

Why self-healing only for bounded cases? Automated recovery is useful only when the blast radius is controlled. Safe actions include retrying a failed connector, restarting a stateless worker, quarantining a bad batch, or temporarily serving the last known good dataset. Anything involving schema changes, security/privacy concerns, or repeated failure should hand off to manual intervention.

Why include AI quality controls in an SRE document? NIST's AI RMF says AI systems should be designed, deployed, and evaluated with trustworthiness considerations in mind. For this use case, malformed JSON, low confidence, hallucination risk, and language-detection failure affect operational reliability because bad AI output can create bad business decisions.

B. Keyword explanations and formulas

Term	Meaning / Calculation
SLI	Service Level Indicator. The metric you monitor, such as pipeline success rate or dashboard availability.
SLO	Service Level Objective. The target level for an SLI, such as 99.5% monthly availability.
Pipeline success rate	$\text{successful scheduled runs} \div \text{total scheduled runs}$. If 99 out of 100 scheduled runs succeed, the success rate is 99%.
Data freshness	The elapsed time between expected data arrival and the time the processed output becomes available to users.
Analytics availability	$\text{successful dashboard or API requests} \div \text{total dashboard or API requests}$.
p50 / median	The middle value. Half of requests are faster than this and half are slower.
p95	The 95th percentile. Ninety-five percent of requests are as fast as or faster than this value.
Null rate	$\text{null values in required fields} \div \text{total records in the batch}$.
Duplicate rate	$\text{duplicate records} \div \text{total records in the batch}$, based on a stable key or hash.
Schema drift	A source changes column names, required fields, or data types without warning.
Validation pass rate	$\text{rules passed} \div \text{total validation rules executed}$ for a batch or job run.
Fallback dataset	The most recent validated dataset that can be safely served while the latest batch is under investigation.
Quarantine table	A holding table for bad or suspect records so that one broken batch does not contaminate published analytics.
RCA	Root-Cause Analysis. A structured review of what failed, why it failed, why it was not caught earlier, and what control must be improved.
CAP	Corrective Action Plan. The specific fix, owner, due date, and validation metric created after the RCA.
RACI	Responsible, Accountable, Consulted, Informed. A matrix that clarifies role ownership during delivery and incidents.

C. Reference links

- Google SRE Book - Service Level Objectives:
<https://sre.google/sre-book/service-level-objectives/>
- NIST SP 800-61 Rev. 3 - Incident Response Recommendations and Considerations for Cybersecurity Risk Management: <https://csrc.nist.gov/pubs/sp/800/61/r3/final>
- NIST AI Risk Management Framework:
<https://www.nist.gov/itl/ai-risk-management-framework>

- Azure Monitor overview:
<https://learn.microsoft.com/en-us/azure/azure-monitor/fundamentals/overview>
- Application Insights overview:
<https://learn.microsoft.com/en-us/azure/azure-monitor/app/app-insights-overview>
- Log Analytics overview:
<https://learn.microsoft.com/en-us/azure/azure-monitor/logs/log-analytics-overview>
- Azure Monitor alerts overview:
<https://learn.microsoft.com/en-us/azure/azure-monitor/alerts/alerts-overview>
- Azure Monitor action groups:
<https://learn.microsoft.com/en-us/azure/azure-monitor/alerts/action-groups>
- Monitor Azure Data Factory:
<https://learn.microsoft.com/en-us/azure/data-factory/monitor-data-factory>
- Monitor Azure Blob Storage:
<https://learn.microsoft.com/en-us/azure/storage/blobs/monitor-blob-storage>
- Azure Container Apps jobs: <https://learn.microsoft.com/en-us/azure/container-apps/jobs>
- Azure Key Vault overview:
<https://learn.microsoft.com/en-us/azure/key-vault/general/overview>
- P&G 2025 Annual Report - Superiority across product, package, brand communication, retail execution and value:
<https://us.pg.com/annualreport2025/superiority-across-product-package-brand-communication-retail-execution-and-value/>

PPT Guide

Yes — for a **4-minute Operations Manager presentation**, I would build a **full ops section**, but only **present the highest-value visuals** and keep the rest as backup or appendix.

What you need to make for your part

Core slides you should definitely prepare

- **1. Role and objective slide**
 - Title: **SRE / Operations Manager**
 - Show:
 - your role in one sentence
 - the reliability objective
 - the 3 core priorities: **availability, freshness, trustworthiness**
 - Visual:
 - a simple 3-icon row or 3-pill layout
- **2. SLI / SLO slide**
 - Use your table with:
 - **Pipeline Success Rate**
 - **Data Freshness**
 - **Analytics Availability**
 - Visual:
 - table only
 - highlight the SLO values in bold
 - Keep this very clean because it is one of your strongest slides
- **3. Monitoring and alerting architecture slide**
 - This should answer:
 - **what are we monitoring**
 - **what tools are doing it**
 - **how alerts are sent**
 - Visual:
 - **architecture diagram**
 - Diagram flow:
 - **Lazada / Sources**
 - **Python Ingestion / ETL**
 - **Azure Data Lake / Storage**
 - **AI Processing**
 - **Web App / Dashboard**
 - then below or beside that:
 - **Azure Monitor**
 - **Application Insights**
 - **Log Analytics**
 - **Azure Alerts / Action Groups**
 - Add one small callout:

- **Email / SMS / webhook-based escalation**
 - Azure Monitor is the central monitoring layer, Application Insights captures app and API telemetry, Log Analytics is the log query layer, and Action Groups handle notifications and automated actions. Azure Data Factory is used for orchestrating and scheduling data workflows, and Azure Storage monitoring covers storage availability, performance, and resilience. ([Microsoft Learn](#))
- **4. Operations Dashboard slide**
 - Show this as a **prototype screen or wireframe**
 - Include only the most important cards:
 - API latency
 - Error rate
 - Pipeline success rate
 - Data freshness
 - Null / duplicate spikes
 - Dashboard availability
 - Incident status
 - Self-healing action log
 - Visual:
 - dashboard mockup
 - This is the best place to show the “proactive operations” idea
- **5. Incident response playbook slide**
 - Visual:
 - **pipeline-stage incident table**
 - Keep only 5 rows:
 - Source/API
 - Data lake
 - ETL/validation
 - AI inference
 - API/dashboard
 - This slide should show:
 - incident
 - detection rule
 - auto action
 - manual handoff
 - This is where you prove you are not just monitoring, you also know what to do next
- **6. RACI slide**
 - Visual:
 - RACI matrix
 - Roles:
 - Operations Manager
 - Data Engineer
 - AI Engineer
 - Project Manager

- Tasks:
 - detect and declare severity
 - restore pipeline
 - restore AI output
 - send status updates
 - approve closure
 - complete RCA / CAP
 - This is important because it shows operational ownership, not just technical metrics
 - **7. Risk assessment slide**
 - Visual:
 - **risk matrix** or **2-column table**
 - Left side:
 - schema drift
 - null spikes
 - duplicate spikes
 - malformed JSON
 - prompt degradation
 - stale dashboards
 - missing competitor mapping
 - Taglish misclassification
 - Right side:
 - control / mitigation
 - This helps connect your section to the actual dataset and the rest of the product
 - **8. Backlog and improvement roadmap slide**
 - Visual:
 - **priority table** or **Now / Next / Later**
 - Example:
 - **Now:** freshness alerts, publish gates, fallback dataset, uptime checks
 - **Next:** schema-drift rules, AI confidence gates, duplicate/null controls
 - **Later:** runbook automation, capacity dashboards, AI-assisted ops
 - This makes your section feel practical and mature
-

Backup / appendix slides you should still prepare

These may not all be spoken, but they help in Q&A.

- **Severity model slide**
 - P1 to P5
 - support window
 - acknowledge target
 - restore target

- **RCA / CAP slide**
 - timeline
 - trigger
 - failed control
 - why not detected earlier
 - corrective action
 - validation metric
 - **Keyword explanation slide**
 - p50
 - p95
 - p99
 - schema drift
 - malformed JSON
 - last known good dataset
 - fallback dataset
 - quarantine batch
-

Best diagrams for your section

1. Monitoring architecture diagram

Make this one for sure.

Layout

- top row:
 - Lazada / source feeds
 - Python scripts / ETL
 - Azure Data Lake / storage
 - AI extraction layer
 - Web dashboard
- bottom row:
 - Azure Monitor
 - Application Insights
 - Log Analytics
 - Azure Alerts / Action Groups

What it communicates

- where failures happen
- where metrics are collected
- how alerts are generated

2. Incident pipeline diagram

Use a left-to-right flow.

Flow

- Source/API
- Landing
- ETL
- AI inference
- API
- Dashboard

Under each box, place:

- sample incident
- sample auto action
- sample handoff trigger

This is a very strong visual for a 4-minute talk.

3. Operations Dashboard mockup

Even if fake, this helps a lot.

Cards to include

- p95 API latency
 - error rate
 - pipeline success rate
 - freshness status
 - null spike alert
 - incident severity
 - self-healing action log
 - alert queue
-

4. Risk matrix

Use:

- **Impact** on one axis

- **Likelihood** on the other

Put your risks inside:

- schema drift
 - malformed JSON
 - stale dashboard
 - duplicate spike
 - token spike
 - missing feed
-

5. Prioritization chart

Use:

- **business impact**
- **recurrence**
- **detectability gap**
- **effort**

You can show:

- a simple scoring formula
 - or a 2x2 matrix
-

What to prepare for each slide

For every slide, prepare these 3 things:

- **What this slide shows**
- **Why it matters**
- **What action it supports**

That keeps your talk clean and prevents rambling.

Example for the Operations Dashboard slide:

- what this slide shows: the live reliability view of the platform
 - why it matters: lets the team detect issues before business users are affected
 - what action it supports: alerting, self-healing, and escalation
-

What to prepare as prototype material

Since you do **not** have time to build the real thing, the smartest move is a **conceptual web app prototype**.

Best prototype scope for your part

Build only these screens:

- **Operations Dashboard**
- **Incident Detail page**
- **Runbook / Escalation panel**
- optional: **Alert History page**

What the prototype should fake

- fake telemetry cards
- fake incident list
- fake alert banners
- fake self-healing log
- fake fallback / retry status
- fake Azure-connected labels

What it should not try to do

- real Azure authentication
- real Action Groups integration
- real SMS sending
- real pipeline execution
- real AI remediation

How to describe it honestly

Say:

- this is a **high-fidelity front-end prototype**
- it **simulates** Azure-connected monitoring behavior
- live integrations are **conceptual for the proposal stage**

That is fair and believable.

How Codex can help you build the prototype

Codex is an AI coding agent that helps write, review, and ship code faster, and can be used either with local tools or by delegating work in the cloud. ([OpenAI Help Center](#))

Best way to use Codex here

Ask it to build:

- a **React web app**
- with a **dashboard layout**
- fed by **mock JSON data**
- styled like an enterprise admin console
- with pages for:
 - Operations Dashboard
 - Incident Details
 - Runbook Panel
 - Alert History

What to tell Codex

Give it a prompt like:

- build a React prototype for an Azure-style operations dashboard
- use mock data only
- include cards for p95 latency, error rate, data freshness, null spikes, dashboard availability, self-healing actions, and incident status
- include an incident table with severity, owner, affected component, and recovery target
- include a runbook panel and escalation section
- make it look like a real internal enterprise monitoring app

Best conceptual positioning

Do **not** say:

- “this is fully integrated with Azure”

Say:

- “this is a concept prototype of the proposed Azure-connected operations interface”

That keeps you safe.

Opportunity slide: where AI can help operations

Yes, there is a good opportunity slide here.

But I would **not** present AI as the primary self-healing engine yet.

Why

For operations, first-line recovery should stay:

- deterministic
- script-based
- reversible
- auditable

That makes it safer.

Better AI opportunity framing

Title:

- **Future Opportunity: AI-Assisted Operations**

Show 3 boxes:

- **Current**
 - rule-based alerts
 - scripted self-healing
 - manual RCA and escalation
- **Next Opportunity**
 - AI incident summarization
 - AI root-cause suggestions
 - AI runbook recommendation
 - AI alert clustering / deduplication
 - AI RCA draft generation
- **Guardrail**
 - AI suggests actions
 - humans or scripts execute approved recovery
 - no autonomous high-risk production changes

If you want to connect it to Gemini

You can say:

- Gemini is already used in other project areas
- in operations, the better fit is **AI-assisted triage and analysis**
- not direct autonomous recovery yet

That makes the slide feel connected to the rest of the team without overclaiming your scope.

Best speaking flow for your 4 minutes

You said you'll manage the time yourself, so this is just the best coverage flow.

- start with **reliability objective**
- move to **3 SLIs / SLOs**
- show **monitoring architecture**
- show **Operations Dashboard**
- show **incident playbook**
- end with **risk + backlog + AI opportunity**

That tells a full operations story:

- what success is
 - how you measure it
 - how you watch it
 - how you respond
 - how you improve it
-

Final checklist: what you need to make and prepare

Slides

- title slide for your role
- SLI / SLO table slide
- monitoring architecture slide
- Operations Dashboard mockup slide
- incident response table slide
- RACI matrix slide
- risk assessment slide
- backlog / prioritization slide
- optional AI opportunity slide
- appendix slides for severity model, RCA/CAP, and keyword explanations

Visuals

- one architecture diagram
- one pipeline incident flow
- one dashboard wireframe or mockup
- one RACI table
- one risk matrix

- one backlog / roadmap chart

Content

- final 3 SLIs / SLOs
- final alert routing wording
- final self-healing wording
- final RACI
- final risk list
- final backlog priorities
- final AI opportunity framing

Prototype prep

- decide on 3 to 4 screens only
- use mock JSON data
- keep it front-end only
- label it as conceptual
- use Codex to generate the React structure and sample telemetry UI ([OpenAI Help Center](#))

Q&A prep

- explain p95 in one sentence
- explain schema drift in one sentence
- explain why self-healing is scripted first
- explain why AI is advisory in ops, not full autonomous recovery
- explain why Azure Monitor + App Insights + Log Analytics + Alerts fit your design ([Microsoft Learn](#))

If you want the next step, I'd turn this into a **slide-by-slide PPT build guide** with exact titles, bullets, and suggested visuals for each slide.

[Mieka] Product Management

1st artifact

Project charter

Infographic - one pager

2nd artifact

Gantt chart

Business value - always put numbers

Initial Feature Ideas

Feature ideas dump

All ideas here are suggestions for features built off of the initial solutions in the [ALL] notes tab
:DD you may choose to use the idea or tweak it according to what's feasible on your end.

Product name: [we don't have one yet]

Different screens

Intelligence Command Center screen

the agentic signals screen **i'm not sure if we'll be including something like this**

- Brand switching early warning: counts verified repeat buyers (verifiedPurchase = true) whose reviewRating dropped to 1–2 stars within the last 14 days, grouped by brand; multiplied by average order value to estimate revenue at risk in pesos
- Share shift predictor: computes a 3-week rolling average star rating per competitor brand; flags any competitor gaining more than +0.2 stars in a 2-week window as a market share threat
- Promo opportunity signal: ARM co-occurrence mining on reviewText keywords; surfaces brand+theme pairs where confidence exceeds 80%
- Revenue-at-risk estimate: derived by multiplying at-risk buyer count by average product price
- Autonomous decision log: timestamped record of every agent action: ingestion events, schema patches, model refreshes, ARM recomputes, and escalation triggers

Not on this screen but still on the agentic side:

More informal suggestion based on what was discussed:

- Under every single metric card, a recommended course of action may pop up depending on current data. Sample below:
 - Sample format in UI
 - Recommended action: Generate retention promo for Pantene 400ml verified buyers
 - Agent confidence: 91% · Based on: 47 flagged buyers, scent complaint cluster, Rejoice bundle signal active
 - [Approve and generate] [Modify parameters] [Dismiss]
- If their review sentiment stabilizes or improves, the agent logs this as a successful intervention and increases its confidence weighting for similar future signals
- If sentiment continues to decline despite the promo, the agent surfaces a new signal: "Retention promo did not stop sentiment decline. root cause may be formulation, not price. Escalating to R&D."

Brand Overview Dashboard (Marketing, R&D, Product supply)

Market Operations

- Real-time review velocity: count of new reviews ingested in the last 24 hours grouped by brand and subcategory; derived from reviewDateISO filtered to today;

flags any brand seeing a sudden spike or drop in review volume as a market activity signal

- Issue priority queue: ranks active problems by a composite urgency score = (severity weight × volume); severity is derived from reviewRating (1-star = 3, 2-star = 2, 3-star = 1) multiplied by the count of reviews mentioning the same theme in the last 7 days; top issues are shown as a ranked list with recommended owner (store, distributor, or field team) based on attribution tag
- Competitor in-market alerts: flags when a competitor brand's review volume spikes more than 40% week-over-week in a specific subcategory; derived by comparing this week's competitor review count vs the prior 3-week average; interpreted as a competitor promotion or launch event
- Clearance/promo response signal: cross-references reviews mentioning price or sale keywords (sale, promo, discount, flash sale, lazada sale) with reviewRating trends; if promo-tagged reviews score lower than non-promo reviews for the same product, flags a value-perception problem that market ops should address
- Fastest-moving issues this week: catches emerging problems before they become dominant; derived by comparing this week's theme keyword frequencies vs last week's for the active category

Research and Development

- Product complaint taxonomy: classifies reviewText into a structured complaint hierarchy using keyword matching:
 - Formulation (formula, ingredients, chemicals, harsh),
 - Scent (bango, amoy, fragrance, smell, kulang, masyadong malakas),
 - Texture (consistency, malapot, malagkit, runny, watery, thick),
 - Performance (effectiveness, works, hindi gumagana, walang epekto),
 - Usability (pakete, dispenser, pump, mahirap buksan, leaks),
 - Durability (lasts, tagal, nauubos agad, mabilis maubos);
 - each category shows volume and average rating
- Noise separation layer: filters out reviews tagged as courier or seller attribution before any product analytics are computed; R&D only sees reviews classified as product feedback, so courier damage complaints never inflate the "packaging" complaint count
- Unmet needs detector: scans for "I wish", "sana", "kung may", "wish it had", "mas gusto ko sana", "if only" patterns in reviewText; groups these by product and theme; surfaces as a ranked list of expressed consumer desires that no current product is satisfying
- Repeat complaint tracker: identifies themes that appeared in the top complaints 3+ months ago and are still appearing today; signals that a previously known issue has not been resolved; derived by comparing monthly theme frequency over the full reviewDateISO range
- Ingredient mention extractor: scans reviewText and ingredients column for specific ingredient mentions (sulfate, paraben, silicone, natural, pH balanced,

dermatologist); maps positive vs negative sentiment context for each ingredient mention; gives R&D a direct consumer perception signal per ingredient

Product Supply

- Store vs distributor vs field signal separation: the attribution classifier extends beyond product/seller/courier to flag distribution-related keywords (out of stock, wala sa tindahan, unavailable, hindi available, kulang) as distributor signals, and in-store display keywords (shelf, display, nakita sa mall) as retail execution signals
- Formulation feedback by SKU: maps the R&D complaint taxonomy (scent, texture, performance, etc.) down to individual SKU level using productName matching; supply teams can compare, for example, Downy 500ml vs Downy 900ml vs Downy 1L to see if a specific size variant has disproportionate complaints
- Packaging and usability signals: specifically isolates reviews mentioning packaging attributes (bottle, pack, sachets, dispenser, seal, leaks, pabula, natanggal) and scores them separately from product performance signals;
- Volume-weighted complaint priority: weights each complaint not just by count but by numberOfReviews on that product listing; a complaint on a 500-review product is more statistically significant than the same complaint on a 10-review product; the weighting ensures supply teams prioritize issues with proven scale
- Urgency-to-fix ranking: ranks all active product issues by a fix priority score
- New vs known issue classifier: flags whether each surfaced issue is new (not present in reviews older than 30 days) or persistent (present across multiple months)

Competitor Intelligence Screen

- Brand rating trend chart : 4-week rolling average of reviewRating per brand, plotted as a line chart; each brand is its own line; gaps indicate no reviews that week
- Share shift delta : difference between a competitor's current 4-week avg rating and their prior 4-week avg rating; shown as +/- pts per brand
- Competitor gap analysis : themes that appear frequently in a competitor's 4-star+ reviews but rarely in P&G's equivalent product reviews; derived by comparing top positive keyword sets between brand groups
- Company-brand-product mapping : each productName is matched against a known brand dictionary (P&G brands list vs competitor map); unmatched brands are flagged as "unclassified" for manual review
- Listing Velocity — Time to Traction (new): measures days from dateAddedToCatalog to first review per product; shown as a bar per brand; flags P&G SKUs with >90-day silence vs. competitors pulling reviews within 30–50 days
 - surfaces as a retail visibility or search ranking gap
- Competitor Launch Detection (new): alerts when a competitor's dateAddedToCatalog is within the last 60 days AND review count already exceeds a threshold (e.g. 20+

reviews); rendered as a "New Entrant" card with brand name, SKU, days since listing, current review count, and early avg rating

Other features:

- 5 Vectors of Superiority Scores: reviewText is scanned for theme keywords mapped to each of P&G's 5 vectors
 - Product → effective/works/quality;
 - Packaging → box/bottle/damaged;
 - Communication → label/claims/ingredient;
 - Retail execution → price/promo/stock;
 - Value → sulit/worth/mahal;
 - vector score = positive keyword hits / total keyword mentions for that vector

Other filters:

- Sector chip : maps to the category column in the dataset; selecting "Fabric Care" filters all rows where category contains fabric-related values
- Sub-category chip: maps to the subcategory column; "Fabric Enhancer" further filters within the sector
- Date range: filters on reviewDateISO; default is last 30 days

Insights to action ideas (agentic side):

- R&D ticket generator automatically converts clusters of similar complaints into structured product improvement briefs routed to the right team

Gen AI layer

- Every metric card, every KPI has a small "what this means" layer built in - a one sentence/two sentence interpretation of the data
- Automatic insight buttons to press to get more info on a specific metric's data

As for SRE operations..

Operations Dashboard Screen

- **API latency tracker:** computes p50, p95, and p99 response time per API endpoint in 15-minute windows; flags warning when p95 exceeds 2 seconds and critical when p95 exceeds 5 seconds for 2 consecutive windows
- **Error rate monitor:** computes failed requests ÷ total requests per service in the last 15 minutes; flags when 5xx error rate exceeds 2% or when failed requests exceed 50 in a single window
- **Pipeline success rate:** computes successful scheduled runs ÷ total scheduled runs per day for each ingestion and ETL job; flags when daily success rate drops below 95% or when the same critical pipeline fails 2 runs in a row

- **Retry volume tracker:** counts retry attempts per pipeline run in the last 24 hours; flags when retries exceed 3 for the same job because this suggests an unstable source, transformation step, or sink
- **Processing delay monitor:** computes $\text{publishTimestamp} - \text{sourceArrivalTimestamp}$ per dataset; flags when delay exceeds 30 minutes for near-real-time feeds or 4 hours for batch feeds
- **Data freshness monitor:** computes current time minus latest successful dataset timestamp per table, API, or dashboard feed; marks data as stale when freshness exceeds the assigned SLA such as 1 hour for live feeds or 24 hours for daily datasets
- **Record count variance check:** compares current batch row count against the prior 7-day average for the same source; flags when volume changes by more than $\pm 20\%$ without a known promo, campaign, or launch event
- **Null rate monitor:** computes $\text{null values} \div \text{total records}$ for required fields per batch; flags when null rate exceeds 5% or doubles versus the prior 7-day baseline
- **Duplicate rate monitor:** computes $\text{duplicate records} \div \text{total records}$ using primary key or hash matching per batch; flags when duplicate rate exceeds 1%
- **Schema drift checker:** compares incoming column names, data types, and required fields against the expected schema before publish; flags added columns, removed columns, renamed columns, and type mismatches
- **Validation pass rate:** computes $\text{passed validation rules} \div \text{total validation rules}$ per job run; flags when pass rate drops below 98% or when any required validation rule fails
- **Source completeness monitor:** checks whether all expected files, partitions, or API pages arrived for the scheduled load; flags when delivered volume falls below 95% of expected input
- **AI classification health:** tracks uncategorized rate, confidence score average, and sentiment distribution per day; flags when uncategorized volume exceeds 10%, average confidence drops below 0.70, or negative sentiment rises by more than 15 percentage points day over day
- **Language detection failure monitor:** computes $\text{records with unknown or failed language tags} \div \text{total text records}$ per batch; flags when failure rate exceeds 3%
- **Prompt quality tracker:** tracks fallback responses, retry rate, tool-call failure rate, and low-rating responses per 100 chatbot sessions; flags when fallback rate exceeds 10% or retry rate exceeds 15%
- **Token usage monitor:** computes average input tokens, output tokens, and total token usage per 100 AI requests; flags when total token usage rises more than 25% above the prior 14-day average
- **Dashboard availability monitor:** computes $\text{successful dashboard loads} \div \text{total dashboard load attempts}$ in the rolling 30-day window; flags when availability falls below 99.5%
- **Widget failure tracker:** computes $\text{failed widget calls} \div \text{total widget calls}$ per dashboard in the last hour; flags when any card, chart, or filter exceeds a 3% failure rate
- **Dashboard response time monitor:** computes page load time and widget render time per session; flags when page load exceeds 4 seconds or when any critical widget render exceeds 2 seconds

- **Incident command panel:** shows open incidents with severity level, affected service, assigned owner, start time, current status, workaround, and target recovery time
- **Self-healing action log:** records automated retries, worker restarts, quarantine routing, and fallback dataset activation; stores timestamp, trigger condition, action taken, and result
- **Runbook and escalation panel:** maps each alert rule to severity level, response target, escalation owner, and linked recovery steps

Keyword Explanations

- **p50 / median response time:** the middle response time; 50% of requests are faster than this and 50% are slower.
- **p95 / 95th-percentile response time:** 95% of requests are this fast or faster; the slowest 5% are worse than this
- **p99 / 99th-percentile response time:** 99% of requests are this fast or faster; this helps show rare worst-case slowdowns
- **API endpoint:** a specific API route or function being called
- **15-minute window:** the metric is measured using requests collected within the last 15 minutes
- **2 consecutive windows:** the threshold must be breached twice in a row before it is flagged

[1] Initial Product Vision, Roadmap, and Features

P&G's Consumer IQ

PRODUCT VISION

In the next 3 years, Consumer IQ will give P&G brand teams, executives, and functional departments prescriptive, agentic consumer intelligence with capabilities such as automatically surfacing brand switching risks, competitor market shifts, and promo opportunities directly from insights based on scraped and processed online product review data, reducing insight-to-decision time from 4–6 weeks to under 24 hours across all six P&G sectors.

ROADMAP

	JAS		OND		JFM	AMJ
Data Foundation	Feature 1 ETL Pipeline	Feature 2 Brand-Product Mapping	Feature 3 Attribution Classifier			
	Enabler 1 Airflow Orchestration	Enabler 2 Schema Validation			Feature 4 Multilingual NLP	
	Enabler 3 PII Masking + Privacy Compliance Layer					

PRODUCT FEATURES

Consumer IQ is a consumer intelligence and operations system built for P&G teams across Marketing, R&D, Product Supply, and Data Engineering organized across four major screens.

- The *Intelligence Command Center* serves as the agentic nerve center that surfaces brand switching early warnings, share shift predictions, promo opportunity signals, and revenue-at-risk estimates backed by an autonomous decision log and a recommended action layer that suggests and learns from interventions in real time.
- The *Brand Overview Dashboard* is the day-to-day workhorse split into three lanes where Market Operations tracks review velocity and competitor in-market alerts, R&D gets a noise-separated view of product complaints taxonomized by formulation and performance plus unmet needs detection and ingredient sentiment mapping, and Product Supply monitors distribution signals and

SKU-level complaint breakdowns all scored against P&G's 5 Vectors of Superiority and filterable by sector, subcategory, and date range.

- The *Competitor Intelligence Screen* plots rolling brand rating trends and share shift deltas against a structured company-brand-product map to show where rivals are gaining ground.
- The *Operations Dashboard* gives Data Engineering full visibility into API latency, pipeline health, data quality checks, AI model performance, and dashboard reliability with an incident command panel and self-healing action log built in. Across every screen a Gen AI interpretation layer translates every metric into a plain-language insight so any team member can understand the signal and act on it.

[2] Product Vision and Strategy

P&G's Consumer IQ

PRODUCT VISION STATEMENT

In the next three years, Consumer IQ will transform how P&G brand teams and executives respond to the market, moving from periodic research cycles to always-on, agentic intelligence that automatically detects brand switching risks, competitor shifts, and promo opportunities from real consumer signals, compressing insight-to-decision time from weeks to under 24 hours across all six P&G sectors.

BUSINESS GOALS

- **Operational Efficiency.** Cut insight-to-decision time from 4 to 6 weeks to under 24 hours across all six P&G sectors.
- **Revenue Protection.** Detect brand switching risks early enough to deploy retention interventions before at-risk buyers churn.
- **Competitive Responsiveness.** Shorten the window between a competitor move and P&G's counter-response with near real-time market signals.
- **Product Improvement Velocity.** Surface clean SKU-level complaint signals and unmet needs continuously to accelerate R&D decision-making.
- **Cross-functional Alignment.** Give Marketing, R&D, Product Supply, and Data Engineering a single shared source of consumer intelligence.
- **Platform Scalability.** Build the foundational infrastructure to expand ConsumerIQ to other channels and markets over three years.

VALUE PROPOSITION/KPIs

- **Business Value.** Faster intervention on brand switching risks protects revenue. Early competitor detection improves promo timing and market share defense. Clean, SKU-level product signals reduce wasted R&D cycles.
- **Key Measurable Benefits KPIs.** Insight-to-decision time reduction, revenue-at-risk identified and recovered, number of agent-recommended actions approved and resolved, reduction in uncategorized or noisy review data, and dashboard availability uptime across all six sectors.

USER PERSONAS

Role	Pain Point	Goal
Brand Manager	Insight cycles take 4 to 6 weeks. By the time data arrives, the window to respond has already closed.	Catch brand health risks early and act on them before they become revenue problems.
R&D Scientist	Courier and seller complaints that pollute product feedback make it hard to isolate genuine formulation signals.	Understand exactly what consumers dislike about a product formulation and why, down to the ingredient level.
Supply Chain Lead	Complaint data aggregated at the brand level makes it hard to pinpoint whether the issue is in a specific size variant, a region, or the supply chain itself.	Identify which SKUs have real product issues versus distribution or shelf execution problems, and prioritize fixes by scale.
Market Ops Manager	Competitor signals are scattered and slow. Promo effectiveness is usually measured after	Detect competitor promotions and launches early, and validate whether P&G's own

	the campaign has already ended.	promos are actually working.
P&G Executive	Reports from different teams use different data sources and arrive at different times, making it hard to form a single clear picture.	Get a fast, reliable read on brand portfolio health and competitive position without needing to dig into raw data.
Data Engineer	Issues in the pipeline such as schema drifts, null rate spikes, AI classification degradation, often go unnoticed until a business user flags bad data downstream.	Keep every pipeline, model, and data feed running reliably so that the business users always have accurate, fresh data.

- Product Vision Statement
- Business Goals
- Value Proposition/KPIs
- User Personas
-

Raw data view, role based dashboards, configurable dashboards, data scraping
Scrape, to clean, publish data link

Agentic

Self healing agent

ETL

Looking to improve themselves

Ex. new ads new attributes. Without interfering ETL. An agent can fix this

Agentic Analytics

Chatbot

Product Vision statement: unifying fragmented feedback from every digital touchpoint into a single, agentic platform that surfaces actionable insights across the 5 Vectors of Superiority, enabling brand teams to act faster, smarter, and with greater confidence than any competitor.

Business Goals

- Accelerate time-to-insights
- Unify the consumer signal
- Build a self-healing, scalable data pipeline
- Democratize consumer intelligence
 - Configurable dashboard views that teams can personalize without engineering support
-

Value Propositions

KPIs

- Insight latency: timestamp comparison between scrape and publish
- Pipeline uptime
- Self-healing resolution rate:
- Dashboard active users:
- Chatbot query satisfaction
- Time to act on alert:

User Personas

Timings - 3-6 months

Benefits - get the numbers for example 5 minute. Think about the numbers, KPIs,

Quick analytics

COSTING

- Compare cost of tokens
- Data lakehouses, serverless computing lambda
- Azure
- How much AI would cost,

[3] Product Features and Roadmap

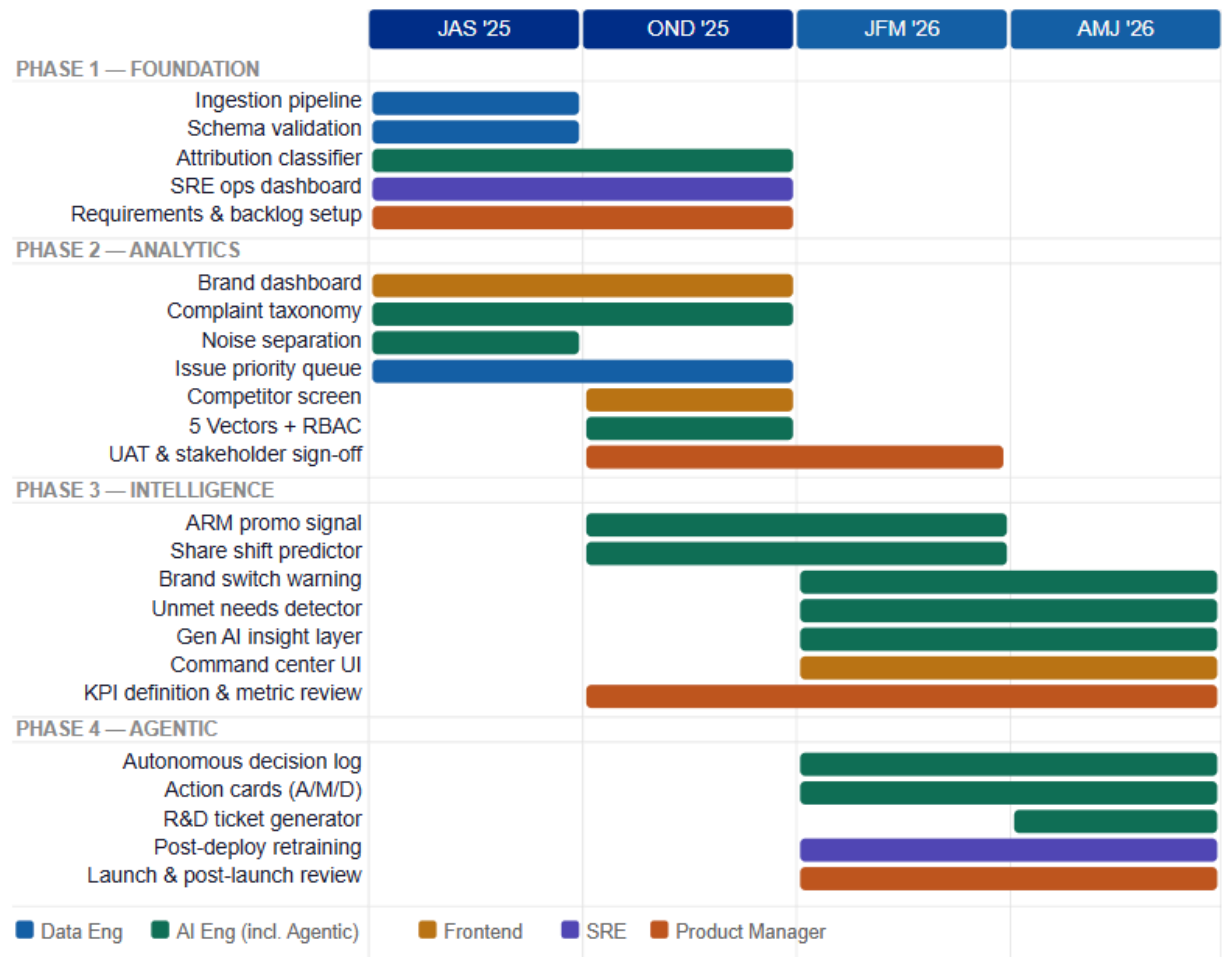
P&G's Consumer IQ

TIMINGS

Go Product Roadmap (not yet sure with this one, will try working on the gantt chart)

	Release 1	Release 2	Release 3	Release 4
DATE	JAS	OND	JFM	AMJ
NAME	Foundation	Analytics Core	Intelligence Layer	Agentic Platform
GOAL	Establish a reliable, validated data pipeline that ingests reviews daily with zero manual intervention	Deliver brand, R&D, and supply teams their first self-serve dashboards with accurate complaint and competitor data	Equip marketing and strategy teams with predictive signals: share shifts, promo opportunities, and competitor gaps	Enable autonomous, auditable AI recommendations with human approval gates across all business teams
FEATURES	Ingestion pipeline; Schema drift & null rate monitors; Attribution classifier (product / seller / courier); Language detection & NLP pre-processing; SRE Operations Dashboard v1	Brand Overview Dashboard; Product Complaint Taxonomy (scent, texture, performance, etc.); Noise separation layer; Issue Priority Queue; Sector / subcategory / date filters	Competitor Intelligence Screen; 5 Vectors Superiority Score; Share Shift Predictor; ARM Promo Opportunity Signal; Role-based access control (RBAC)	Intelligence Command Center; Brand Switching Early Warning + Revenue-at-Risk; Gen AI "What This Means" layer; Autonomous Decision Log; Approve / Modify / Dismiss action cards; R&D Ticket Generator
METRICS	Daily ingestion success rate ≥99%; Null rate <5%; Attribution accuracy ≥90%; p95 API latency <2s	Dashboard load time <4s; Complaint taxonomy coverage ≥85% of reviews; UAT sign-off from Marketing, R&D, and Supply leads	ARM confidence threshold ≥80% on surfaced signals; Competitor brand map covers top 10 fabric care brands; RBAC enforced across all roles	Agent action approval rate ≥70%; R&D brief generation time reduced ≥60%; Model retraining cadence fully automated monthly

Gantt Chart version



TEAM INVOLVEMENT

Team	Responsibilities	Key Deliverables	Phases	Collaboration Points
Digital Product Manager	Own product vision, backlog prioritization, stakeholder alignment, define success metrics, manage UAT and launch readiness	Roadmap, PRDs, JTBD definitions, UAT sign-off, launch plan, post-launch review	1-4	All teams; Marketing, R&D, Supply (business stakeholders)
Data Engineering	Build and maintain ingestion pipelines; schema validation; data lake management; ETL orchestration	Pipeline monitors, schema drift checker, record count variance, null/duplicate rate monitors	1-4	AI Eng (data contracts), SRE (monitoring hooks)
AI Engineering	Build NLP classifiers	Attribution	2-4	Data Eng (training

	(attribution, taxonomy, ARM, sentiment); train and version all ML models; deploy Gen AI and agentic layer	classifier, complaint taxonomy, unmet needs detector, ARM promo signal, 5 Vectors Score, LLM 'what this means' layer		data), DPM (feature requirements)
SRE/DevOps	Platform reliability, CI/CD pipelines, uptime monitoring, incident response, post-deployment model retraining automation	Operations Dashboard, alert rules, self-healing log, runbook panel, dashboard availability monitor	1, 2, 4	Data Eng (pipeline health), AI Eng (model monitoring)

ANSWER KEY JTBDS

- Timings (what features to deploy per quarter) - Gantt Chart
- Team involvement (parts to play by other team members)
- UI Prototype
- Answer key JTBDs

Raw data view, role based dashboards, configurable dashboards, data scraping

Scrape, to clean, publish data link

Agentic

Self healing agent

ETL

Looking to improve themselves

Ex. new ads new attributes. Without interfering ETL. An agent can fix this

Agentic Analytics

Chatbot

[lan] AI Engineering

This isn't actually building the entire system flawlessly; it's proving that we understand the logic, the business value, and the architecture.

Output:

1. AI Model Architecture (1 page)
2. AI Methodology (MLOps) - Summary Report (2 pages)
 - a. Feature Engineering
 - b. Model Training/Experimentation
 - c. Model Analysis/Evaluation
 - d. Model Deployment
3. AI Output
 - a. i.e., Insights, Visualization, Chatbot, etc.
 - b. With Potential Prototype/Demo
 - c. Must be tied to business needs

How do we summarize consumer feedback to derive actionable insights for brand-building efforts for the following vectors:

- Product
 - Performance (Formulation): Identifying consistency issues. For example, the review stating “Tubig na po yung fabcon.. Hindi na tulad ng dati na thick yung consistency” tells us a competitor’s product quality is degrading.
- Packaging
- Communication
- Retail Execution
 - Spotting a delivery or seller issue. For example, “legit xia... slamat po sa seller” indicates a verified organic seller experience.
- Value
 - Identifying price sensitivity. For example, “Great value for money” or “2 liters nabili ko lang sa halagang 111 pesos”.

Must Have/be answered:

1. What's your prompt summary?
2. How will this benefit the company?
3. Is it scalable? If yes, how and why?

AI Draft Plans:

1. Must have continuity from the Data Engineering Role (4 Roles meron: Digital Product Manager, Data Engr, AI Engr, SRE/Operations Manager)
2. Sentiment analysis based on consumer reviews/feedback for specific products and subcategories
 - a. Using that sentiment analysis, we would try to extract business intelligence from our brand/competitors on what is lacking, which strategies we could adapt, and which promotions we could create. (Perhaps using ARM)

- b. Likewise, Company-Brand-Product Mapping/Classification should also be extracted.
3. Review/Feedback Classification (if for the product, for the seller, for the courier—the review is)
4. Product/Review image analysis: text extraction (this idea is still vague)
5. Self-healing AI adapting to the changes from data sources, for such structures, attributes, data types, etc.

System Flow & Components

1. Data Ingestion & Engineering Layer (DE)
 - a. Ingestion: Azure Data Factory
 - b. Storage: Azure Data Lake Storage (& SQL Database)
 - c. Pre-Processing & Compliance: Data Scrubbing and Pre-processing
2. AI Processing Engine
 - a. Compute: Containerized Python Scripts (Azure Container Apps) continuously poll the SQL for new, unprocessed reviews
 - b. Payload Management: Word-Based Truncation
 - c. Model:
 - d. Agentic Extraction: Prompt Engineering
 - i. Sentiment
 - ii. Vector Mapping
 - iii. Action Required Flag
 - iv. Churn Risk & Competitor Mentioned
3. Storage & Serving Layer
 - a. Structured JSON to SQL DB again
4. Business Output
 - a. Decision Support
 - b. Dashboard
 - i. Sentiment Trends
 - ii. Aggregates vector-specific issues
 - iii. Show high-priority supply chain & competitor churn alerts

[1] Output Document

1-Page Architecture

Input: Where is the data coming from? (Lazada)

Engine: What processes it? (Python script)

Storage: Where does it go? (A clean database; specify)

Output: Who uses it? (UI; specify, agent; specify)

2-Page MLOps Report

Feature Engineering

Outline:

(Data Preparation & Transformation)

- Problem: E-commerce reviews are unstructured, multilingual (Taglish), and contain noise (emojis, slang)
- Approach: Python to drop nulls, deduplicate records, and handle basic text cleaning.
- Automated Extraction: djfsjghsf

Output:

Traditional NLP feature engineering struggles with the nuanced, code-switching nature of Philippine Taglish for such complex, unstructured e-commerce feedback. Therefore, our methodology utilizes Agentic LLM Extraction via the Gemini API. The AI model performs the feature engineering by parsing the unstructured text and outputting strictly formatted JSON aligned with P&G business needs.

1. DE-AI Continuity

The Data Engineer handles the foundational data cleaning and PII scrubbing, staging the clean data in a unified SQL database.

2. AI Payload & Token Management

Instead of re-cleaning the data, the AI pipeline executes "Payload Management". Before text reaches the LLM, the Python script truncates excessively long reviews to a strict token limit. This engineered constraint reduces API inference costs without losing the core consumer sentiment.

3. Feature Extraction

The LLM dynamically extracts specific business vectors, isolating the 5 Vectors of Superiority. This also uses predictive metrics like "Churn Intent", extracting explicit mentions of consumers switching to competitors.

Model Training/Experimentation

Outline:

- Model Selection: Justify why we chose a pre-trained LLM API (like Gemini) instead of training a custom sentiment model from scratch. Reason: Speed to market, zero-shot capability on Taglish, and the ability to process unstructured text into JSON instantly.
- Experimentation (Prompt Tuning): Explain that "training" in this context actually means "Prompt Engineering." Describe how you iteratively tested prompts to ensure the AI correctly identified nuances (e.g., distinguishing a complaint about a Courier vs. a complaint about Product Formulation).
- Future Iteration (RAG/Fine-Tuning): Note that as the dataset grows, you could use Retrieval-Augmented Generation (RAG) to feed the AI-specific P&G product manuals or historical data to improve its context.

Output:

Given the velocity of e-commerce data and the complexity of consumer feedback, we opted for a foundational AI model via the Gemini API.

- Iterative Prompt Engineering
“Training” shifted to rigorous Prompt Engineering. We experimented extensively with prompt structures to eliminate AI hallucinations and enforce deterministic JSON formatting.
<insert prompt structure>
- Predictive Intelligence
Initial experimentation focused primarily on sentiment. However, retroactive sentiment lacks business urgency. We iterated our prompt to include an Action Required Trigger, looking for supply chain defects, and a Churn Risk metric, transforming the model from a passive summarizer into a proactive business agent.

Model Analysis/Evaluation

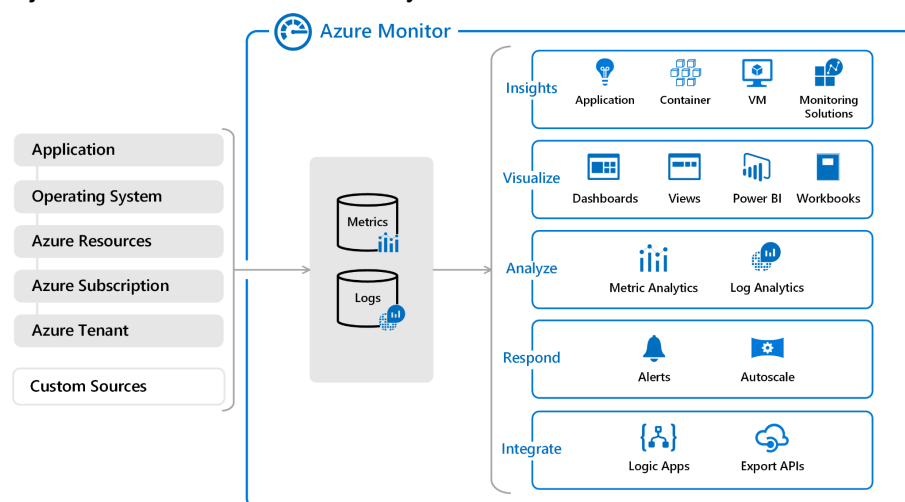
Outline:

- Evaluation Metrics: How do you know the AI is right? For classification (Sentiment and Vector mapping), you can use traditional metrics like Accuracy, Precision, and Recall on a manually labeled test set of 100 reviews.
- Business Alignment Check: The ultimate evaluation metric is business value. Is the AI correctly flagging the Action Required alerts for the Supply Chain or Retail teams?

Output:

Evaluating an Agentic LLM pipeline requires blending deterministic software testing with business-aligned validation.

- Deterministic Pipeline Validation:
To prevent pipeline failures, we engineered Python try/except mechanisms. If the API times out or the JSON structure breaks, the system logs the error to “” and injects a safe fallback dictionary.



- Human-in-the-Loop (HITL) Auditing

Accuracy is evaluated via HITL sampling. An engineer periodically audits a random sample of AI's output to calculate the precision and recall of the Vector Mapping and Churn alerts.

- **Business Metric Alignment**
The evaluation metric is the reduction of Time-to-Insight (TTI). By automating categorization, the model reduces the time it takes for "who" to identify a defect or issue.

Model Deployment

(CI/CD Pipeline)

Outline:

- **Architecture Flow:** Web Scrapers > Cloud Storage (Data Lake) > Python AI Processing Script > Structured Database > BI Dashboard
- **Scalability & Automation (Docker/Airflow):** Explain that the Python scripts will be containerized using Docker to ensure they run anywhere. A tool like Apache Airflow will schedule the script to run daily at midnight to capture new reviews.
- **Reliability & "Self-Healing":** Address what happens if an API fails or the e-commerce site changes its layout. Implement try/except blocks and logging mechanisms to alert the data engineering team if the scraper breaks. Mention tracking API costs and latency.

Output:

While the prototype utilizes a local environment for demonstration, the enterprise production scaling strategy adheres to standard CI/CD and MLOps practices within the Azure ecosystem.

- **Containerization & CI/CD**
The Python extraction scripts and dashboard will be containerized using Docker. Deployment pipelines will be managed via GitHub Actions or Azure DevOps, pushing images to the Azure Container Registry.
- **Secure Execution**
The data processing engine will run on Azure Container Apps, triggered by scheduled jobs.
- **Enterprise Security**
API keys, database connection strings, and credentials will be strictly managed via Azure Key Vault, adhering to zero-trust security principles.
- **Continuous Monitoring**
// consult with SRE/OP

The system will utilize “what” to actively monitor API latency, execution failures, and token costs, alerting the engineering team to any “Data Drift” or prompt degradation.

[2] Hybrid Engine

Executive Summary

To address the sheer volume and linguistic complexity of unstructured e-commerce feedback, traditional machine learning models and large language model APIs both present distinct operational flaws. To solve this, our team engineered a Hybrid AI Pipeline. This architecture layers a high-speed, zero-cost Lexicon NLP Engine (Layer 1) with an Agentic LLM (Layer 2). By implementing a dynamic routing mechanism based on “Lexicon Confidence”, the system filters out standard noise, while escalating complex, slang-heavy complaints to the LLM for deep semantic extraction, ensuring accurate business intelligence at a fraction of standard API costs.

AI Model Architecture

AI Methodology

I. Feature Engineering

1. Layer 1: Deterministic Lexicon Engine

The pipeline first processes scrubbed data via a proprietary Python Lexicon. This engine uses bilingual sentiment weights, intensifier multipliers, and a negation window to mathematically score standard feedback. It maps standard keywords directly to P&G's 5 Vectors of Superiority.

2. Engineered Metric: The Lexicon Confidence Score

To prevent the Masking Effect, Layer 1 calculates a “Lexicon Confidence Score” or Token Hit Rate by dividing the number of recognized lexicon words by the total number of meaningful words in the sentence. If the score drops below 50%, the engine refuses to guess and routes the review to Layer 2.

3. Layer 2: Agentic LLM Extraction (Gemini API)

Low-confidence or highly critical reviews are escalated to the Generative AI. The LLM bypasses rigid keywords to perform deep semantic feature extraction, strictly outputting JSON payloads for Sentiment, Vector, Action Required, and Churn Risk.

Use Case: TOD000

II. Model Training & Experimentation

Given the rapid evolution of Gen Z e-commerce slang, static dictionaries become obsolete quickly. Instead of retraining a custom neural network, we designed an Active Learning Loop (Data Flywheel).

1. Prompt Engineering & Deterministic Fallbacks

Experimentation focused on refining the LLM’s prompt to ensure strict JSON adherence. To guarantee pipeline stability, we engineered Python try/except fallbacks. If the API times out, the system injects a safe fallback dictionary to prevent dashboard crashes.

2. The Self-Updating Lexicon

To ensure the Layer 1 engine becomes smarter over time, the architecture includes a scheduled batch process. Monthly, the Agentic LLM analyzes the “unknown slang” it processed and automatically appends new trending words to the Python script’s SENTIMENT_LEXICON.

Use Case: TODDOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOO

III. Model Analysis/Evaluation

Evaluating this hybrid architecture relies on a combination of unit economics, deterministic software testing, and business-value metrics.

1. Unit Economics & Scalability
2. Human-in-the-Loop (HITL) Auditing
3. Business Value Delivery

IV. Model Deployment

While the Minimum Viable Product (MVP) utilizes a local SQLite database for rapid prototyping and seamless DE-to-AIE handoffs, the deployment strategy is designed for Microsoft Azure to align with P&G’s corporate IT standards.

- Containerization & Orchestration
- Secure Storage
- Zero-Trust Security
- Continuous Monitoring

DE-AIE Continuity

Unified Table

- DE Columns (The Foundation): Review_ID, Source (Lazada/Shopee), Date, Raw_Taglish_Text, Brand, etc.
- AIE Columns (The Enrichment): AI_Sentiment, PG_Vector, Churn_Risk, Competitor_Mentioned, etc.

Data Engineer

- ETL Pipeline: How are you scraping Lazada automatically?
- Data Model: ER Diagram showing how a product is linked to a review and a brand, etc.
- Data Quality & Privacy: For Compliance and Data privacy, we should explain how we scrub Personally Identifiable Information (PII) before AIE works with the data.

AI Engineer

- Take the clean dataset/structured data from DE
- MLOps Pipeline: Explain how our AI script is containerized (Docker???) and pulls data daily to update the insights
- Output: present the dashboard, emphasize DE's robust data pipeline underneath the dashboard

Unified Diagram

DE

- Web Scrapers (Azure Functions) > Raw Data Storage (Azure Blob Storage) > Data Cleaning & PII Scrubbing (Python or ...) > Structured Database

AIE

- AI Engine (Extract insights from Structured DB) > Analytics Database (Stores AI tags) > Agentic UI (Dashboard)

Data Engineering

[1] Data Pipeline Architecture

[2] Methodology

[3] Data Output

